

Architecture IoT

Voici l'architecture technique complète de bout en bout pour FutureKawa, incluant la logique de code à chaque étape .

Étape 1 : Le Terrain (IoT - Côté ESP32)

C'est ici que le monde physique devient numérique.

- **Le matériel** : Le capteur **DHT22** est branché sur l'**ESP32**.
- **Le code (C++ dans Arduino IDE)** :
 1. **Setup** : Le code connecte l'ESP32 au réseau Wi-Fi, puis le connecte au serveur MQTT (l'adresse IP de ton PC).
 2. **Lecture** : Toutes les 30 secondes, le code interroge le capteur pour obtenir la température et l'humidité.
 3. **Encodage** : Le code formate ces deux chiffres en une phrase JSON :

```
{"temperature": 28.5, "humidite": 60.2} .
```
 4. **Envoi** : Le code publie ce message sur un canal spécifique, par exemple `futurekawa/BR/entrepot/1/mesures` (BR pour Brésil).

Étape 2 : Le Transport (Le Broker MQTT)

C'est le "facteur" de l'architecture.

- **L'outil** : **Eclipse Mosquitto** (qui tourne silencieusement dans ton Docker sur le port 1883).
- **L'action** : Il ne fait rien d'autre que recevoir le message de l'ESP32 et le redistribuer instantanément à tous ceux qui sont "abonnés" à ce canal.

Étape 3 : L'Ingestion & Stockage (Backend Pays)

C'est le cerveau local, qui archive les données pour un pays donné.

- **Les outils** : Ton API **FastAPI** (Python) et ta base de données **PostgreSQL** (Docker).
- **Le code (Python)** :
 1. **L'écouteur (Subscriber)** : Un script Python tourne en tâche de fond. Il est abonné au canal `futurekawa/+/entrepot/+/mesures` . Dès que Mosquitto lui transmet le JSON de l'ESP32, il le lit.
 2. **L'écriture (SQLAlchemy)** : Il convertit ce JSON et fait une requête SQL automatique (INSERT) pour sauvegarder la température, l'humidité et l'heure exacte dans la table `mesures` de PostgreSQL.

3. **L'API REST** : En parallèle, FastAPI expose des "routes" (ex: `GET /lots/1/mesures`) pour permettre à d'autres applications de venir lire cette base de données.

Étape 4 : La Consolidation (Backend Central)

C'est la tour de contrôle du siège social.

- **L'outil** : Une deuxième API **FastAPI** (Python).
- **Le code (Python avec `httpx`)** : Il agit comme un "Proxy" (un intermédiaire). Quand on lui demande les données du Brésil, il ne va pas chercher dans une base de données locale. Il fait lui-même une requête HTTP vers l'API du Backend Pays (Étape 3), récupère la réponse, et la renvoie proprement. C'est ça, une architecture distribuée !

Étape 5 : L'Affichage (Frontend)

L'interface visible par le directeur de FutureKawa.

- **Les outils** : **React** (Vite), **Tailwind CSS**, et **Chart.js**.
- **Le code (TypeScript/JavaScript)** :
 1. **La requête (Axios)** : Quand l'utilisateur clique sur un lot de café, le code React demande au Backend Central (Étape 4) de lui fournir l'historique des mesures.
 2. **L'affichage (Chart.js)** : Les données reçues sont injectées dans le composant graphique, qui trace la courbe des températures et de l'humidité à l'écran.

Étape Bonus : La Simulation (Pour la démo)

Puisque tu n'as qu'un seul vrai capteur physique (pour le Brésil), tu vas tricher intelligemment pour remplir ton interface.

- **Le code (Script Python de test)** : Un petit fichier de 20 lignes génère des chiffres aléatoires et les envoie sur Mosquitto pour l'Équateur (`futurekawa/EC/entrepot/...`) et la Colombie (`futurekawa/CO/entrepot/...`). Le système traitera ces fausses données exactement comme les vraies !

Tableau de Connexion (Pinout)

Le câblage entre le capteur **DHT22** (module à 3 broches) et l'**ESP32** est très simple et ne nécessite que 3 câbles (jumpers) :

Broche du Capteur (DHT22)	Câble (Couleur recommandée)	Broche de l'ESP32	Fonction
VCC (ou <code>+</code> / <code>V</code>)	 Rouge	3V3 (ou <code>3.3V</code>)	Alimentation électrique
DATA (ou <code>OUT</code> / <code>S</code>)	 Jaune ou Vert	GPIO 4 (ou <code>D4</code>)	Transmission des données

Broche du Capteur (DHT22)	Câble (Couleur recommandée)	Broche de l'ESP32	Fonction
GND (ou  / G)	 Noir	GND	Mise à la terre (fermeture du circuit)

Instructions de Montage étape par étape

Pour ne pas abîmer le matériel et avoir un rendu visuel propre pour la soutenance, voici comment procéder sur la plaque blanche :

1. **Placement de l'ESP32** : Enfiche l'ESP32 tout en haut de la breadboard. **Attention** : Il doit être placé "à cheval" sur la rainure centrale qui sépare la plaque en deux. Cela permet d'avoir accès aux trous de chaque côté des broches de l'ESP32 pour y brancher les câbles.
2. **Placement du DHT22** : Enfiche le capteur un peu plus bas sur la breadboard, avec la grille orientée vers toi.
3. **Le Câblage** : * Relie un câble entre la ligne de la broche **VCC** du capteur et un trou correspondant à la broche **3V3** de l'ESP32.
 - Fais de même pour relier le **GND** au **GND**.
 - Relie la broche **DATA** du capteur à la broche numérique **D4** (ou GPIO 4) de l'ESP32.

 **Le piège classique (à noter pour le chef de projet)** : > Si l'école vous fournit un capteur DHT22 "nu" (qui possède **4 broches** au lieu de 3 et n'est pas soudé sur une petite carte), il faudra obligatoirement ajouter une résistance de **10kΩ** entre la broche VCC et la broche DATA pour que le signal soit lisible. Si c'est un module à 3 broches (comme ton ancien capteur bleu), la résistance est déjà intégrée, vous n'avez besoin de rien d'autre !